

TuntiTutka

Paperinen työpaketti

Työaikaseuranta mainostoimistolle: Svelte, Express, SQLite ja julkaisu tuotantoon

18 työviikkoa · päivätön aikataulu · luovutus 18. työviikon perjantai

Tämä paketti on aikataulu ja tarkistuslista tilanteisiin, joissa sivusto ei ole auki. Projektipäiväkirja kirjoitetaan sivustolla ja viedään repositoryn project-docs-kansioon. Rasti tässä vihossa ei ole palautus: työ on aina Git-repositoryssa.

Aikataulu on päivätön: työviikko 1 on se viikko, jolla opiskelija aloittaa, ja projekti kestää 18 työviikkoa.

Julkiseen repositoryyn ei laiteta henkilötietoja, koulun tunnisteita eikä muiden nimiä. Tekijänimestä sovitaan ohjaajan kanssa.

Aikataulu

Vko	Ajoitus	Viikon aihe	Vaihe
1	Työviikko 1 / 18	Aloitus	A
2	Työviikko 2 / 18	Suunnitelma ja tietomalli	A
3	Työviikko 3 / 18	Julkaistu runko	A
4	Työviikko 4 / 18	Kirjautuminen ja roolit	A
5	Työviikko 5 / 18	Tuntikirjaus	A
6	Työviikko 6 / 18	Hallintanäkymät	B
7	Työviikko 7 / 18	Omat kirjaukset haltuun	B
8	Työviikko 8 / 18	Yhteenvetojen laskenta	B
9	Työviikko 9 / 18	Raporttinäkymä ja kaaviot	B
10	Työviikko 10 / 18	Asiakaskatselmointi	B
11	Työviikko 11 / 18	Palautemuutos	C
12	Työviikko 12 / 18	Mobiili ja saavutettavuus	C
13	Työviikko 13 / 18	Tietoturva	C
14	Työviikko 14 / 18	Testausviikko	C
15	Työviikko 15 / 18	Laatu ja dokumentointi	C
16	Työviikko 16 / 18	Julkaisuehdokas ja julkaisutestaus	D
17	Työviikko 17 / 18	Julkaisu v1.0	D
18	Työviikko 18 / 18	Näyttö	D

Palautus viimeistään 18. työviikon perjantai. Sivusto: tehtävien tarkat ohjeet, toteutusavut ja projektipäiväkirja.

Vaihe A – Sovelluksen ydin: suunnitelma, julkaistu runko, roolit ja tuntikirjaus

Työviikko 1 / 18 – Aloitus

Kehitysympäristö ja julkinen repository ovat valmiit, ja Svelte + Vite -sovellusrunko käynnistyy omalla koneella.

- Asenna työkalut (Node LTS, VS Code, Git) ja kirjaa versiot README:hen.
- Luo GitHub-repository ja tee julkisen repon tarkistukset: yksityisyys, tekijänimi ja alaikäisen huoltajan suostumus ohjaajan kautta.
- Luo Svelte + Vite -projekti ja käynnistä kehityspalvelin.
- Lue toimeksianto, kirjoita kysymyslista ohjaajalle ja luo kansiorakenne. Sitten ensimmäinen commit ja push.

Valmis kun: Repositoryssa on käynnistytvä Svelte-runko ja README, jossa on käynnistyskomennot; ohjaaja on kuitannut julkisen repon tarkistuslistan; kysymyslistassa on vähintään kuusi kysymystä.

Työnäyte Git-repositoryyn ennen rastia: commit-historian alku, kuvakaappaus dev-palvelimesta selaimessa, kuitattu julkisuustarkistuslista ja kysymyslista (p1, k1; s12 alkaa).

Työviikko 2 / 18 – Suunnitelma ja tietomalli

Hyväksytty tekninen suunnitelma: priorisoidut käyttäjätarinat, tietomalli, tietovarastoverailu, rautalangat ja issue-taulu työmääräarvioineen.

- Kirjoita käyttäjätarinat hyväksymiskriteereineen ja priorisoi ne P0/P1/P2 ohjaajan kanssa.
- Piirrä tietomalli ja kirjaa periaate: yhteenvetosummaa ei tallenneta, ne lasketaan kirjauksista.
- Vertaa kolmea tietovarastoa (SQLite, JSON-tiedosto, PostgreSQL) ja perustele valinta omalla datallasi.
- Piirrä rautalangat kirjaus- ja raporttinäkömästä ja pilko P0 issueiksi työmääräarvioineen.

Valmis kun: project-docs/suunnitelma.md on repossa ja ohjaaja on hyväksynyt rajauksen kirjatulla kommentilla; jokaisella P0-tarinalla on issue, hyväksymiskriteerit ja arvio tunteina.

Työnäyte Git-repositoryyn ennen rastia: suunnitelma.md, tietomallikaavio, vertailutaulukko, issue-taulu ja ohjaajan kirjattu hyväksyntä (s1, s4, s5, s8, p8; s6 alkaa arvioista).

Työviikko 3 / 18 – Julkaistu runko

Tyhjä sovellusrunko on julkisessa osoitteessa: Express tarjoaa Svelte-buildin, /api/health vastaa ja SQLite-skeema syntyy skriptillä.

- Rakenna Express-palvelin, joka tarjoaa Svelte-buildin ja vastaa reitillä /api/health; kytke Viten dev-proxy.
- Luo SQLite-tietokanta ja skeema ajettavalla init-skriptillä.
- Vertaile julkaisualustoja (Render, Fly.io, Railway), valitse ja perustele valintasi.
- Julkaise runko, kirjaa julkinen osoite README:hen ja kirjoita k2-selvitys.

Valmis kun: /api/health palauttaa 200 julkisessa osoitteessa toisen henkilön selaimella testattuna; init-skripti luo skeeman tyhjään tietokantaan; k2-selvitys ja alustaperustelu ovat repossa.

Työnäyte Git-repositoryyn ennen rastia: julkinen osoite README:ssä, julkaisulokin kuvakaappaus, init-skripti, k2-selvitys ja alustavertailu (k2, s14 ensijulkaisu; s9 pohjustuu).

Työviikko 4 / 18 – Kirjautuminen ja roolit

Työntekijä ja projektipäällikkö kirjautuvat sisään ja näkevät roolinsa mukaisen navigaation; API-reitit on suojattu middlewarella.

- ☐ Toteuta käyttäjätaulu bcrypt-hasheineen ja siemendata: yksi projektipäällikkö ja kaksi työntekijää.
- ☐ Vertaa evästesessiota ja tokenia, tee päätös ohjaajan kanssa ja kirjaa keskustelu.
- ☐ Toteuta kirjautuminen, uloskirjautuminen ja roolin mukainen navigaatio.
- ☐ Suojaa API-reitit autentikointi- ja roolimiddlewarella.

Valmis kun: Väärä salasana antaa selkeän suomenkielisen virheilmoituksen; työntekijä ei näe hallintanavigaatiota; kirjautuminen säilyy sivun päivityksessä; suora API-kutsu ilman kirjautumista palauttaa 401.

Työnäyte Git-repositoryyn ennen rastia: kirjattu vertailu ja yhteinen päätös, kuvakaappaukset molemmilla rooleilla ja middleware-koodin commit (p9; s7 osa).

Työviikko 5 / 18 – Tuntikirjaus

Työntekijä kirjaa tunnit (projekti, tehtävälaji, tunnit, päivä, selitys) ja näkee omat kirjauksensa listana. Kirjausnäky on rakennettu itse alusta ilman valmista käyttöliittymäkirjastoa.

- ☐ Toteuta kirjauslomake rautalangan mukaan omina Svelte-komponentteina ja kirjaa komponenttijako.
- ☐ Toteuta POST /api/kirjaukset backend-validointineen (tunnit 0,25–24, pakolliset kentät, olemassa olevat viitteet).
- ☐ Toteuta omien kirjausten lista (GET /api/kirjaukset/omat).
- ☐ Testaa virhesyötteet (0 h, negatiivinen, 25 h, ei-numeerinen, puuttuva tehtävälaji) ja kirjaa tulokset.

Valmis kun: Kirjaus tallentuu SQLiteen ja näkyy listassa sivun päivityksen jälkeenkin; virheellinen syöte ei tallennu ja käyttäjä näkee suomenkielisen virheviestin kentän vieressä; kirjaus onnistuu alle 30 sekunnissa. Ota aika.

Työnäyte Git-repositoryyn ennen rastia: komponenttijakokaavio, kuvakaappaus lomakkeesta ja listasta, validointitestien tulostaulukko ja commitit (p6, s9, k3; k5 toteutus alkaa).

Vaihe B – Ominaisuudet: hallinta, omat kirjaukset, lasketut yhteenvedot ja asiakaskatselmointi

Työviikko 6 / 18 – Hallintanäkymät

Projektipäällikkö perustaa projekteja, hallitsee projektityyppejä ja tehtävälajeja sekä liittää työntekijöitä projekteihin, ja sovellus on jaettu näkyviin reitityskirjastolla.

- ☐ Vertaa kahta reititysratkaisua, valitse ja perustele; jaa sovellus reitteihin.
- ☐ Toteuta projektien CRUD eli luonti, luku, muokkaus ja poisto hallintanäkymään (nimi, tyyppi, aikaväli, kuvaus).
- ☐ Toteuta tehtävälajien ja projektityyppien hallinta kevyenä: lisäys sekä nimen ja kuvauksen muokkaus.
- ☐ Toteuta projektin jäsenyys: työntekijän lisäys projektiin ja poisto projektista.

Valmis kun: Työntekijän kirjauslomakkeessa näkyvät vain projektit, joihin hänet on liitetty; vain projektipäällikkö pääsee hallintanäkymiin (testattu myös suoralla URL-osoitteella); reittikartta on dokumentoitu.

Työnäyte Git-repositoryyn ennen rastia: perusteltu kirjastoalinta package.json-diffillä, reittikartta ja demo molemmilla rooleilla (p7, k4; s7 laajenee).

Työviikko 7 / 18 – Omat kirjaukset haltuun

Työntekijä muokkaa ja poistaa omia kirjauksiaan, suodattaa niitä viikon ja projektin mukaan ja näkee oman viikkosummansa; omat yhteystiedot voi päivittää. Oma tuntikirjanpito TuntiTutkassa alkaa.

- ☐ Toteuta kirjauksen muokkaus ja poisto: omistajuus tarkistetaan backendissa.
- ☐ Toteuta viikko- ja projektisuodatus omien kirjausten listaan ja kirjaa valittu viikkokäytäntö.
- ☐ Laske oma viikkosumma kyselyllä (ei tallenneta) ja toteuta yhteystietojen päivitys.
- ☐ Aloita omien projektityötuntien kirjaaminen TuntiTutkaan tehtävälajeittain.

Valmis kun: Toisen käyttäjän kirjausta ei voi muokata edes suoralla API-kutsulla (testi kirjattu: odotettu 403, saatu 403); viikkosumma täsmää käsin laskettuun; omia kirjauksia on sovelluksessa vähintään viikon verran.

Työnäyte Git-repositoryyn ennen rastia: API-testin tulos (403), viikkosumman käsinlaskuverailu, kuvakaappaus omista kirjauksista ja arvio vs. toteuma -taulukon alku (p7, s6, s7).

Työviikko 8 / 18 – Yhteenvedojen laskenta

Raportti-API laskee työtunnit lennossa kolmella ryhmittelyllä (viikoittain, henkilöittäin ja tehtävälajeittain) eikä yhtään summaa tallenneta; projektipäällikkö näkee taulukkoyhteenvedon ja porautuu yksittäisiin kirjauksiin.

- ☐ Suunnittele raporttireittien vastausmuoto paperilla ennen koodia.
- ☐ Rakenna testiaineisto ja laske odotetut summat käsin ennen toteutusta.
- ☐ Toteuta laskenta omana moduulinaan GROUP BY -kyselyillä aikaväli- ja projektirajauksin.
- ☐ Toteuta projektipäällikön taulukkoyhteenvedo ja porautuminen yksittäisiin kirjauksiin.

Valmis kun: Kolme raporttireittiä palauttaa summat, jotka täsmäävät käsin laskettuihin odotusarvoihin vähintään kolmella eri testiaineistolla; tietokannassa ei ole summataulua ja skeema todistaa sen; laskentamoduulilla on vähintään kolme yksikkötestiä.

Työnäyte Git-repositoryyn ennen rastia: odotusarvotaulukko (kirjattu ennen ajoa), testiajon tulokset, laskentamoduulin commit ja skeeman kuvaus (s7 päätyönäyte, p4; s10 pohjustuu).

Työviikko 9 / 18 – Raporttinäkymä ja kaaviot

Projektipäällikön raporttinäkymä esittää viikkotunnit pylväskaaviona ja tehtävälajijakauman kaaviona; tyhjä data, verkkovirhe ja lataustila käsitellään.

- ☐ Vertaa kaaviokirjastoa (koko, lisenssi, dokumentaatio, saavutettavuus) ja perustele valinta.
- ☐ Toteuta raporttihaku selaimen fetch-funktiolla sekä muunnosfunktio ja sen testi.
- ☐ Toteuta ryhmittelyn vaihto (viikko, henkilö, tehtävälaji) ja aikavälin valinta.
- ☐ Käsittele tyhjä aineisto, verkkovirhe ja lataustila ohjaavin viestein.

Valmis kun: Kaaviot piirtyvät oikeasta datasta ja päivittyvät suodattimista; verkkovirhe (testataan katkaisemalla backend) näyttää viestin eikä tyhjää ruutua; muunnosfunktion testi menee läpi; valinta on perusteltu kirjallisesti.

Työnäyte Git-repositoryyn ennen rastia: kirjastovertailu, muunnosfunktion testi sekä kuvakaappaukset kaavioista ja virhetiloista (s10, k4, k3).

Työviikko 10 / 18 – Asiakaskatselmointi

Ulkopuolinen henkilö asiakkaan roolissa kokeilee julkaistua versiota molemmilla rooleilla; palaute on kirjattu testaajan omin sanoin erillään omasta tulkinnasta, ja muutokset on priorisoitu issueiksi.

- ☐ Valmistele katselmointi: esityslista, kaksi testipolkua, valmiit tunnukset ja testidata.
- ☐ Esittele versio asiakaslähtöisesti ja anna testaajan kokeilla itse ilman opastusta.
- ☐ Kirjaa palaute testaajan omin sanoin ja oma tulkinta erikseen.
- ☐ Priorisoi muutokset ohjaajan kanssa ja kirjaa ne issueiksi hyväksymiskriteereineen.

Valmis kun: Katselmointimuistio (testaajan nimetty rooli, ajankohta, testaajan omat sanat, oma tulkinta, sovitut muutokset) on project-docs-kansiossa ja tärkein muutos on issueina hyväksymiskriteereineen.

Työnäyte Git-repositoryyn ennen rastia: katselmointimuistio, priorisoitu muutoslista ja uudet issuet (s2, s3, p10; s1 täydenty).

Vaihe C – Valmiiksi: palautemuutos, mobiili, tietoturva, testaus ja laatu

Työviikko 11 / 18 – Palautemuutos

Katselmoinnin tärkein muutos on toteutettu omassa haarassa ja yhdistetty pääversioon pull requestilla (PR); vähintään yksi merge-konflikti on ratkaistu hallitusti.

- ☐ Valitse katselmoinnin tärkein muutos ja tarkenna hyväksymiskriteerit issueen.
- ☐ Toteuta muutos ominaisuushaarassa pienin commitein.
- ☐ Tee pull request, katselmoi oma koodisi kommentein ja ratkaise merge-konflikti.
- ☐ Todenna muutos julkaistussa versiossa ja kuittaa katselmointimuistion kohta.

Valmis kun: Pull requestin kuvaus ja keskustelu näyttävät mitä muutettiin ja miksi; muutos on tuotannossa; konfliktin ratkaisu näkyy historiassa; katselmointimuistion kohta on kuitattu linkillä.

Työnäyte Git-repositoryyn ennen rastia: pull request -linkki (PR), merge-commit, ennen/jälkeen-kuvakaappaus ja kuitattu muistio (s13; s12 syvenee, p10 jatkuu).

Työviikko 12 / 18 – Mobiili ja saavutettavuus

Tuntikirjaus onnistuu oikealla puhelimella; sovellus toimii näppäimistöllä, lomakkeilla on label-kytkennät ja kontrastit riittävät. Saavutettavuuspistemäärä on vähintään 90 ja ennen/jälkeen-raportit ovat tallessa.

- ☐ Aja Lighthouse (saavutettavuus ja suorituskyky) ja kirjaa lähtötaso talteen.
- ☐ Korjaa responsiivisuus omilla CSS-taitekohdilla ja testaa kirjaus oikealla puhelimella. Ota aika.
- ☐ Korjaa saavutettavuus: label-kytkennät, otsikkohierarkia, fokusjärjestys, kontrastit ja kaavion tekstivastine.
- ☐ Aja Lighthouse uudelleen ja dokumentoi ennen/jälkeen-vertailu.

Valmis kun: Kirjaus on tehty oikealla puhelimella alusta loppuun alle 30 sekunnissa; koko käyttöpolku kulkee pelkällä näppäimistöllä; saavutettavuuspistemäärä on vähintään 90 ja molemmat raportit ovat repossa.

Työnäyte Git-repositoryyn ennen rastia: Lighthouse-raportit ennen ja jälkeen, kuvakaappaus puhelimelta, korjauslista ja näppäinpolun kuvaus (p6 täydentyy, p3 osa).

Työviikko 13 / 18 – Tietoturva

Tietoturvariskien arviointi on dokumentoitu ja korjaukset tehty: validointi molemmissa päissä, roolipohjainen pääsy jokaisella API-reitillä, salaisuudet ympäristömuuttujissa ja XSS-tarkistus selityskentille.

- ☐ Käy riskilista läpi ja kirjaa arvio: riski → ratkaisu → jäännösriski.
- ☐ Testaa jokainen API-reitti taulukkona: reitti x rooli x odotettu vastaus x saatu vastaus.
- ☐ Korjaa löydökset, siirrä salaisuudet .env-tiedostoon ja tarkista Git-historia.
- ☐ Testaa XSS: kirjauksen selitteeksi skriptitagi. Syötteen pitää näkyä tekstinä.

Valmis kun: Reittitaulukko on ajettu ja jokainen rivi täsmää odotettuun (poikkeamat korjattu ja uusintatestattu); skripti-syöte renderöityy tekstinä; repossa tai sen historiassa ei ole salaisuuksia; arvioidokumentti on project-docs-kansiossa.

Työnäyte Git-repositoryyn ennen rastia: tietoturva-arvio, ajettu reittitaulukko, XSS-testin kuvakaappaus ja .env-järjestelyn commit (s11; s7 täydentyy).

Työviikko 14 / 18 – Testausviikko

Vähintään 12 suunniteltua testitapausta on ajettu kolmessa luokassa (normaali, rajat ja virhetilanteet) ja vähintään kaksi täydellistä virheenkorjausketjua on dokumentoitu.

- ☐ Täydennä testimatriisi vähintään 12 tapaukseen ja kirjaa odotetut tulokset ennen ajoa.
- ☐ Aja testit ja kirjaa havaitut tulokset erillään odotetuista.
- ☐ Korjaa löytyneet virheet täydellisinä ketjuina havainnosta regressiotestiin.
- ☐ Lisää laskentamoduulin yksikkötestit regressiosuojaksi (viikkorajat, tyhjä viikko).

Valmis kun: Testiraportissa jokaisella tapauksella on luokka, odotettu ja havaittu tulos sekä ajankohta; vähintään kaksi ketjua on täydellisiä commit-viittauksineen; regressiotestit ajetaan komennolla ja menevät läpi.

Työnäyte Git-repositoryyn ennen rastia: testiraportti (vähintään 12 riviä), kaksi täydellistä virheenkorjausketjua ja testikoodin commitit (p2, p3; k5 testausosa).

Työviikko 15 / 18 – Laatu ja dokumentointi

Koodi on refaktoroitu testit vihreinä, ja dokumentaatio on valmis: README, käyttöönotto-ohje, riippuvuustaulukko ja asiakkaan pikaohjeet molemmille rooleille.

- ☐ Valitse kolme refaktorointikohdetta, refaktoroi testit vihreinä ja dokumentoi ennen/jälkeen.
- ☐ Kirjoita käyttöönotto-ohje: asennus tyhjään ympäristöön vaihe vaiheelta.
- ☐ Kirjoita asiakkaan pikaohjeet molemmille rooleille ja kokoa riippuvuustaulukko.
- ☐ Kysy lisenssi ohjaajalta; jos vastausta ei vielä ole, kirjaa se avoimeksi asiaksi.

Valmis kun: Regressiotestit menevät läpi refaktoroinnin jälkeen; ennen/jälkeen-diffit ja perustelut ovat repossa; dokumentit ovat repossa ja ohjeet on kirjoitettu käyttäjälle, ei arvioijalle; lisenssin tila on kirjattu.

Työnäyte Git-repositoryyn ennen rastia: ennen/jälkeen-diffit perusteluineen, README, käyttöönotto-ohje, riippuvuustaulukko ja pikaohjeet (p5, k7).

Vaihe D – Julkaisu: julkaisuehdokas ja julkaisutestaus, v1.0 ja näyttö

Työviikko 16 / 18 – Julkaisuehdokas ja julkaisutestaus

Sisältö on jäädytetty, julkaisuehdokas (release candidate, RC) v1.0-rc1 on julkaistu ja merkitty Git-tagilla, ja nimetty ulkopuolinen henkilö on testannut sen molemmilla rooleilla pelkän kirjallisen ohjeen avulla.

- ☐ Tee jäädytyspäättös, luokittele issue't estäviin ja v1.1-listaan ja tarkista tuotantoasetukset.
- ☐ Julkaise julkaisuehdokas ja merkitse se Git-tagilla v1.0-rc1.
- ☐ Asenna sovellus itse puhtaaseen ympäristöön pelkän käyttöönotto-ohjeen avulla ja pidä pöytäkirjaa.
- ☐ Järjestä ulkopuolisen julkaisutestaus ja kirjaa havainnot sekä estävät virheet issueiksi.

Valmis kun: v1.0-rc1 on julkisessa osoitteessa ja merkitty tagilla; oma asennuspöytäkirja ja ulkopuolisen testauspöytäkirja (nimetty rooli, ajankohta, testaajan omat sanat erillään omasta tulkinnasta) ovat repossa; estävät virheet on kirjattu issueiksi (niitä ei korjata kiireellä tällä viikolla vaan seuraavalla).

Työnäyte Git-repositoryyn ennen rastia: julkaisuehdokkaan tagi (v1.0-rc1) ja release, oma asennuspöytäkirja, ohjeen korjausdiffi, ulkopuolisen testauspöytäkirja ja estävien issue-lista (k6, s14 osa, s3 toinen katselmointi).

Työviikko 17 / 18 – Julkaisu v1.0

Julkaisutestauksen estävät virheet on korjattu täydellisenä virheenkorausketjuna, v1.0 on julkaistu ja merkitty tagilla, ja sovellus on luovutettu asiakkaalle.

- ☐ Korjaa työviikon 16 estävät havainnot täydellisenä virheenkorausketjuna.
- ☐ Julkaise v1.0: tag, release ja julkaisutiedote tunnettuine rajoitteineen.
- ☐ Aja savutesti julkaistulle v1.0:lle testipoluilla molemmilla rooleilla ja kirjaa tulos.
- ☐ Kirjoita luovutusviesti asiakkaalle asiakaskielellä: osoite, tunnukset ja pikaohjeet.

Valmis kun: v1.0 on julkisessa osoitteessa ja merkitty tagilla; kolmas ketju on täydellisenä repossa (tai kirjaus siitä, mistä aidosta havainnosta ketju ajettiin); savutestin tulos, julkaisutiedote ja luovutusviesti ovat repossa; viikolle jäi puskuriaikaa eikä mitään uutta aloitettu.

Työnäyte Git-repositoryyn ennen rastia: kolmas virheenkorausketju, v1.0-tagin ja release, savutestin kirjaus, julkaisutiedote ja luovutusviesti (s14, s2; k6 ja p2 täydentyvät).

Työviikko 18 / 18 – Näyttö

Mitään uutta ei rakenneta: näyttöaineisto on täsmälinkitetty kaikkiin 32 vaatimukseen, 8–10 minuutin demo on harjoiteltu ja itsearviointi on kirjoitettu.

- ☐ Täsmälinkit näyttömatriisi: jokaiselle vaatimukselle viikko, työnäyte ja toimiva linkki.
- ☐ Harjoittele 8–10 minuutin demo kellon kanssa toiselle henkilölle.
- ☐ Kirjoita itsearviointi konkreettisiin tilantein: onnistumiset, avun tarve ja mitä tekisit toisin.
- ☐ Tarkista aineiston aukot, luovuta paketti ja pidä puskuri korjauksille.

Valmis kun: Jokainen matriisin rivi osoittaa olemassa olevaan aineistoon ja linkki aukeaa; demo on ajettu kellon kanssa vähintään kerran toiselle henkilölle; itsearviointi sisältää konkreettisia tilanteita, ei yleislauseita.

Työnäyte Git-repositoryyn ennen rastia: valmis näyttömatriisi, demorunko, itsearviointi ja luovutettu paketti (p11 ja koko matriisin koonti).

Viimeiset viisi päivää

Ma · Sisältöjäädytys: viimeinen hyväksytty versio, matriisin täsmälinkitys alkaa

Ti · Aineisto: päiväkirja, AI-loki, testiraportti ja linkkien tarkistus

Ke · Demoharjoitus kellon kanssa (8–10 min) ja itsearvioinnin kirjoittaminen

To · Puskuri: aukkojen korjaus ja tarkistus toisen henkilön kanssa

Pe · Luovutus: näyttömatriisi, projektipäiväkirja, AI-loki ja julkaistu v1.0

Sanasto

Projektin tunnukset ja ammattitermit siinä järjestyksessä, jossa ne tulevat vastaan. Jokainen on selitetty myös sivustolla ensimmäisen käytön kohdalla.

repository vko 1	<i>repo, projektin kansio versionhallinnassa.</i> GitHubissa oleva projektin kansio, jossa koodi, dokumentit ja koko muutoshistoria säilyvät. TuntiTutkan repositoryssa ovat kansiot client/, server/ ja project-docs/.
commit vko 1	<i>yksi tallennettu muutos.</i> Git-historiaan tallennettu muutos ja sen viesti. Yksi commit on yksi looginen muutos, ja commit-historia on itsessään näyttöaineistoa.
frontend	<i>selaimessa toimiva osa.</i> Se osa sovelluksesta, jonka käyttäjä näkee ja jota hän klikkaa. TuntiTutkan frontend tehdään Sveltellä ja Vitellä kansioon client/.
backend	<i>palvelimella toimiva osa.</i> Se osa sovelluksesta, joka ajetaan palvelimella: rajapinta, käyttöoikeudet ja tietokantayhteys. TuntiTutkan backend tehdään Node.js:llä ja Expressillä kansioon server/.
p1, s1, k1	<i>näyttömatriisin vaatimustunnukset.</i> Viikkojen Näytä-riveillä olevat tunnukset viittaavat näyttömatriisin osaamisvaatimuksiin: p on Ohjelmointi, s on Ohjelmistokehittäjänä toimiminen ja k on ohjelmistokomponenttikirjasto. Numero yksilöi vaatimuksen, esimerkiksi s7.
P0 vko 2	<i>pakollinen ydin.</i> Ominaisuudet, joiden on valmistuttava ennen mitään lisäominaisuuksia: kirjautuminen ja roolit, tuntikirjaus, omat kirjaukset ja lasketut yhteenvedot. P0 tehdään ensin.
P1 vko 2	<i>tärkeä jatkosisältö.</i> Sisältö, joka tehdään vasta kun P0 toimii. TuntiTutkassa P1 on kaaviot, mobiilihionta ja yhteenvetojen vienti taulukkotiedostoksi.
P2 vko 2	<i>valinnainen lisä.</i> Sisältö, joka voidaan jättää kokonaan pois ilman että työ jää kesken. TuntiTutkassa P2 on kirjausmuistutus ja usean viikon vertailu.
GitHub-issue vko 2	<i>tehtäväkortti.</i> GitHubissa oleva tehtäväkortti, jossa on hyväksymiskriteerit ja työmääräarvio. Yksi issue on tyypillisesti puolen tai yhden päivän työ, ja siihen viitataan numerolla, esimerkiksi #12.
API vko 3	<i>sovelluksen rajapinta.</i> Palvelimen osoitteet, joita selain kutsuu tietojen hakemiseen ja tallentamiseen, esimerkiksi /api/kirjaukset. TuntiTutkan rajapinta tarkistaa oikeudet itse: käyttöliittymästä piilotettu nappi ei ole suojaus.
JSON vko 3	<i>tekstimuotoinen tietomuoto.</i> Tapa esittää tieto nimi-arvo-pareina tekstinä. TuntiTutkassa JSON on sekä yksi vertailtavista tietovarastoista (JSON-tiedosto) että muoto, jossa rajapinta palauttaa kirjaukset ja raportit selaimelle.
build vko 3	<i>valmiiksi käännetty tuotantoversio.</i> Komennolla tuotettu versio sovelluksesta, joka viedään palvelimelle. Kehityspalvelin ei ole build: julkaistava versio syntyy aina erikseen.
CSV	<i>taulukkotiedostomuoto.</i> Tekstitiedosto, jossa sarakkeet on erotettu toisistaan ja jonka taulukkolaskenta avaa sellaisenaan. TuntiTutkassa yhteenvetojen vienti CSV-tiedostoksi on P1-sisältöä.
middleware vko 4	<i>välikerros.</i> Palvelimen funktio, joka tarkistaa jokaisen pyynnön ennen kuin reitin oma käsittelijä ajetaan. TuntiTutkassa middleware tarkistaa kirjautumisen (401) ja roolin (403).
REST vko 5	<i>rajapinnan osoitetyyli.</i> Tapa rakentaa rajapinta niin, että osoite kertoo kohteen ja HTTP-metodi toiminnon: GET /api/kirjaukset hakee kirjaukset ja POST /api/kirjaukset lisää uuden.
CRUD vko 6	<i>luonti, luku, muokkaus ja poisto.</i> Tietueen neljä perustoimintoa. Projektien CRUD tarkoittaa, että projektipäällikkö voi luoda, listata, muokata ja poistaa projektit itse ilman että tietokantaan kosketaan käsin.
SQL vko 8	<i>tietokannan kyselykieli.</i> Kieli, jolla tietoa haetaan ja muokataan SQLite-kannassa. TuntiTutkan yhteenvedot lasketaan SQL-kyselyllä jokaisella pyynnöllä, eikä summia tallenneta mihinkään.
haara vko 11	<i>branch.</i> Oma kehityslinja, jossa muutoksen voi tehdä rikkomatta pääversiota. Työviikolla 11 palautemuutos tehdään omassa haarassa ja yhdistetään pääversioon vasta katselmoinnin jälkeen.

pull request vko 11	<i>PR, pyyntö yhdistää haara pääversioon.</i> GitHubissa avattava pyyntö, jossa haaran muutokset esitellään, katselmoidaan ja yhdistetään pääversioon. Kuvaus kertoo mitä muutettiin ja miksi. Lyhenne on PR.
XSS vko 13	<i>skriptin ujuttaminen syötteeseen.</i> Hyökkäys, jossa syötekenttään kirjoitettu skripti ajetaan toisen käyttäjän selaimessa. TuntiTutkassa testataan, että kirjauksen selitekenttään kirjoitettu skripti näkyy pelkkänä tekstinä.
T01 vko 14	<i>testitapauksen tunnus.</i> T tarkoittaa testitapausta ja numero yksilöi sen: T01 on ensimmäinen testitapaus ja T02 toinen. TuntiTutkassa tapauksia on 13 (T01–T13), ja jokaisen odotettu tulos kirjataan ennen ajoa.
RC vko 16	<i>release candidate, julkaisuehdokas.</i> Lähes valmis versio, joka julkaistaan ja testataan ennen lopullista julkaisua. TuntiTutkan ensimmäinen julkaisuehdokas on v1.0-rc1 työviikolla 16.
tagi vko 16	<i>versiomerkinä Git-historiassa.</i> Nimilappu, joka kiinnitetään tiettyyn committiin, jotta juuri se versio löytyy myöhemmin: v1.0-rc1 ja v1.0. Release on tagin ympärille tehty julkaisumerkinä kuvauksineen.

Näyttömatriisi – 32 osaamisvaatimusta

Rasti vasta, kun vaatimukselle on täsmällinen työnäyte: linkki, commit, kuva, testirivi tai muistio. Sama työnäyte voi kelvata useaan kohtaan.

Ohjelmointi

- **käyttää ohjelmointieditoria tai kehitysympäristöä** – VS Code, Vite dev -palvelin ja selaimen kehittäjätyökalut käytössä työviikolta 1: versiotaulukko, kuvakaappaus ja npm-skriptit README:ssä.
- **etsii ja korjaa virheitä ohjelmakoodista** – Kolme täydellistä virheenkorjausketjua (työviikot 14, 17; havainnot myös 8–9 ja 16): havainto, toisto, syy, korjauscommit, uusintatesti ja regressiotesti.
- **testaa ohjelman toimintoja** – Testiraportti työviikolta 14: vähintään 12 tapausta odotettuine tuloksineen ennen ajoa; lomakkeet, roolit, tallennus, virhetilanteet ja responsiivisuus testattuina.
- **käyttää rakenteista ohjelmointia toteutuksissa** – Laskentalogiikka omana moduulina työviikolla 8: funktiot, ehdot ja tietorakenteet raporttien ryhmittelyssä; moduulijako client, server, routes ja laskenta.
- **kirjoittaa ylläpidettävää ohjelmakoodia** – Kolme dokumentoitua refaktorointia työviikolla 15 ennen/jälkeen-diffeineen ja perusteluineen; nimeämiskäytäntö ja vastuiden jako kuvattuina.
- **tulkitsee suunnitelmia ja toteuttaa käyttöliittymän tai sen osia** – Kirjausnäkyvä toteutettu työviikolla 5 työviikon 2 rautalangan mukaan itse ilman valmista käyttöliittymäkirjastoa; mobiili ja palautteet käyttäjälle viimeistellään työviikolla 12.
- **tulkitsee suunnitelmia ja toteuttaa ohjelmiston toimintoja** – Hallintänäkymien toiminnot työviikolla 6 käyttäjätarinoiden ja hyväksymiskriteerien perusteella; omien kirjausten hallinta täydentää työviikolla 7. Issue-commit-ketju näkyvissä.
- **sopii tehtävistä tiimin muiden jäsenten kanssa** – Issue-taulu työviikolta 2 alkaen ja viikoittainen sopiminen ohjaajan kanssa tiimiroolissa; tehtävien tila näkyvänä koko projektin ajan.
- **etsii ratkaisuvaihtoehtoja ja ratkoo ongelmia yhdessä tiimin kanssa** – Istuntopapaveritalk työviikolla 4: evästesession ja tokenin hyödyt ja riskit tässä sovelluksessa, kirjattu keskustelu ja yhteinen päätös ohjaajan kanssa.
- **arvioi ratkaisujen toimivuuden yhdessä tiimin kanssa** – Katselmointi työviikolla 10: täyttääkö toteutus käyttäjätarinat ja mitä muutetaan ennen seuraavaa versiota. Muistio ja priorisoitu muutoslista, todennus työviikolla 11.
- **arvioi omaa toimintaa tiimin jäsenenä** – Itsearviointi työviikolla 18 konkreettisin tilantein: mikä onnistui, missä tarvitsit apua ja keneltä, mitä tekisit toisin.

Ohjelmistokehittäjänä toimiminen

- **selvittää kehitystiimin kanssa asiakkaan tarpeet** – Toimeksianto purettu käyttäjätarinoiksi ja käyttäjäryhmiksi työviikolla 2; kysymyslista ja kirjatut vastaukset, rajausta sovittu ja täydennetty työviikolla 10.
- **viestii tekniset asiat asiakaslähtöisesti** – Katselmointiesittely työviikolla 10 ja luovutusviesti työviikolla 17 ilman teknistä jargonia: mitä ratkaisu tarkoittaa käyttäjälle, vaihtoehdot ja rajoitteet.
- **osallistuu version katselmointiin** – Kaksi katselmointia: väliversion asiakaskatselmointi työviikolla 10 ja julkaisuehdokkaan julkaisutestaus työviikolla 16. Palaute ja sovitut muutokset kirjattuina.
- **asettaa kehitystiimin kanssa toteutettavat toiminnot tärkeysjärjestykseen** – P0-, P1- ja P2-priorisointi ohjaajan kanssa työviikolla 2; P0-ydin eli kirjaus, roolit ja raportit toteutettu ensin.
- **jakaa kehitystiimin kanssa toteutettavat toiminnot tehtäviksi** – Käyttäjätarinat pilkottu issueiksi työviikolla 2 hyväksymiskriteereineen, noin puolen tai yhden päivän kokoisina; issue-taulu koko projektin ajan.
- **suunnittelee ja arvioi kehitystiimin kanssa tehtävien toteuttamista** – Työmääräarviot issueissa työviikolta 2 ja arvio vs. toteuma -vertailu työviikolta 7 omista TuntiTutka-kirjauksista; suunnitelman päivitys, kun arvio petti.
- **kehittää ohjelmiston toimintalogiikkaa** – Yhteenvetojen laskenta lennossa työviikolla 8 viikoittain, henkilöittäin ja tehtävälajeittain ilman tallennettuja summia; käyttöoikeudet, validointi ja omistajuussäännöt työviikoilta 4, 7 ja 13.
- **valitsee ohjelmistoon sopivan tietovaraston** – Vertailu SQLite, JSON-tiedosto ja PostgreSQL työviikolla 2 datan rakenteen, käyttötilanteen ja laajuuden perusteella; perusteltu valinta suunnitelmassa.
- **toteuttaa yhteyden tietovarastoon** – Kirjausten ja hallintadatan luku, lisäys, muokkaus ja poisto SQLitestä hallitusti työviikoilta 5–7; skeema ja init-skripti työviikolta 3.
- **hyödyntää rajapintoja ja käsittelee tietoa** – Raportti-API:n kutsu fetchillä työviikolla 9, JSON-muunnos kaavion muotoon testattuna funktiona ja virhetilanteet eli tyhjä data, verkkovirhe ja lataus käsiteltyinä.
- **arvioi ohjelmiston tietoturva** – Tietoturva-arvio työviikolla 13: syötteet, käyttöoikeudet, salasanat, istunnot ja tietojen näkyvyys; ajettu reitti- ja XSS-testi, salaisuudet ympäristömuuttujissa.

- **käyttää versionhallintaa** – Git koko projektin ajan työviikolta 1: tarkoituksenmukaiset commitit, etärepository ja haarakäytäntö. Historia työnäytteenä, koonti työviikolla 11.
- **liittää ohjelman osan olemassa olevaan versioon** – Palautemuutos ominaisuushaarassa työviikolla 11: pull request, itsekatselmointi, konfliktin ratkaisu ja merge pääversioon.
- **julkaisee ohjelman tuotantoympäristöön** – Tuotantobuild, ympäristöasetukset ja julkaisu valittuun pilvialustaan: ensijulkaisu työviikolla 3, julkaisuehdokas 16 ja v1.0 työviikolla 17 julkisessa osoitteessa.

Ohjelmiston toteuttaminen ohjelmistokomponenttikirjastolla

- **ottaa käyttöön ja konfiguroi ohjelmistokomponenttikirjaston käyttöön soveltuvan kehittämisympäristön** – Svelte- ja Vite-projektin luonti ja konfigurointi työviikolla 1: vite.config, dev-proxy backendiin työviikolla 3 sekä kehitys- ja tuotantoasetukset.
- **selvittää ohjelmistokomponenttikirjaston tarjoamat mahdollisuudet ja rajoitteet** – Kirjallinen selvitys työviikolla 3: mitä Svelte ja Vite ratkaisevat eli komponentit, reaktiivisuus ja build, mitä eivät eli reititys ja kaaviot, ja mistä puuttuva otetaan.
- **käyttää ohjelmistokomponenttikirjaston tärkeimpiä toimintoja ja työkaluja** – Komponentit, propsit, tapahtumat, lomakesidonnat, ehdollinen renderöinti ja store istuntotilalle osoitettuina kirjausnäkyssä työviikolla 5 ja raporttinäkyssä 9.
- **tuo kehittämisympäristöön ulkoisia komponentteja** – Kaksi perusteltua ulkoista komponenttia: reitityskirjasto työviikolla 6 vertailuineen ja konfigurointeineen sekä kaaviokirjasto työviikolla 9 kokoineen, lisensseineen ja riippuvuuksineen.
- **suunnittelee, toteuttaa ja testaa ohjelmiston ohjelmistokomponenttikirjastoa käyttäen** – Komponenttirakenne ja vastuut suunnitelmassa työviikolla 2, sovellus toteutettu Sveltellä käyttäjätarinoiden mukaan ja omat sekä kirjastoon liittyvät ratkaisut testattuina työviikolla 14.
- **julkaisee ohjelmiston asiakkaan ympäristöön** – Viten tuotantobuild ja julkaisu sovittuun pilviympäristöön: julkaisuehdokas tagilla v1.0-rc1 työviikolla 16 ja v1.0 työviikolla 17.
- **dokumentoi ohjelmiston sovitulla tavalla** – README työviikolla 15: käyttöönotto, käynnistys, riippuvuudet rooleineen ja ympäristöasetukset; lisäksi asiakkaan pikaohjeet molemmille rooleille.

Muista: sivuston rastit ja kentät tallentuvat vain selaimeen. Ne eivät siirry opettajalle eivätkä korvaa Gitissä olevaa työtä.